LESSON 04 アルゴリズム基礎知識 II

配列を用いた最大値・最小値の探索
第4章 キャンパスに図形を描こう

データ走査の基本ロジック習得と、キャンパス描画機能を活用した数値データの視覚的アウトプット

配列要素のループ比較
基本的なデータ巡回手法

Canvas 2D 座標系
グラフィック描画の基礎

グラフ描画ロジック
数値の視覚可視化

OVERVIEW

本日の学習トピック

TOPIC 01

最大値・最小値の探索

先頭要素の仮定とループ比較による配列走査の基本ロジックを理解します。

TOPIC 02

キャンパス図形描画

2D座標系に基づいた矩形・円・線などの各種グラフィック描画手順を学びます。

TOPIC 03

アルゴリズムの可視化

配列の数値を棒グラフで描き、最大値要素をグラフィックで強調表示します。

ALGORITHM CONCEPT

最大値探索の基本思考法

1. 仮の最大値をセット
配列の先頭要素 **array[0]** を暫定最大値 **max** に保持します。

2. 順次要素と大小比較
インデックス **i = 1** から末尾まで順番に比較を実行します。

3. 条件一致で値を更新
array[i] > max であれば **max = array[i]** へ更新します。

擬似コード (Pseudocode)

```
        // 配列の宣言 let array = [25, 42, 18, 91, 63]; let max = array[0]; for (let
i = 1; i < array.length; i++) { if (array[i] > max) { max = array[i]; } }
```

STEP-BY-STEP

最大値探索の実行ステップ

STEP 01

初期設定

先頭値 array[0] を変数 max へ初期代入します。

STEP 02

ループ開始

カウンタ i を 1 から順次インクリメントして走査します。

STEP 03

比較判定

条件式 array[i] > max の真偽値を各要素毎に評価します。

STEP 04

結果出力

全比較完了後、最終的に保持された max を特定・出力します。

EXECUTION TRACE

変数変化のトレース表

対象配列: [25, 42, 18, 91, 63]

インデックス (i)	配列要素 (array[i])	比較式 (array[i] > max)	更新後の max	状態備考
初期状態	array[0] = 25	-	25	先頭値を初期セット
i = 1	42	42 > 25 (True)	42	max 更新
i = 2	18	18 > 42 (False)	42	保持
i = 3	91	91 > 42 (True)	91	最大値発見！
i = 4	63	63 > 91 (False)	91	保持（完了）

CHAPTER 04

キャンパス (Canvas) 描画の基本構造

1

2D 座標系の理解

画面左上 **(0, 0)** を原点とし、右方向が X 軸正、下方向が Y 軸正となります。

2

描画コンテキストの取得

canvas.getContext('2d') を呼び出して描画用オブジェクトを取得します。

3

図形命令の実行

矩形・パス・円・テキストなどをAPI関数命令で直接出力します。



CANVAS COMMANDS

キャンパス図形描画の主要 API

矩形描画 (Rectangle)

`fillRect(x, y, width, height)`

指定位置に塗りつぶし四角形を描画します。棒グラフの棒を出力する際に最適です。

円・パス描画 (Circle / Path)

`arc(x, y, radius, startAngle, endAngle)`

パスの記述開始と終了で真円や円弧を描写します。データポイント強調に使用できます。

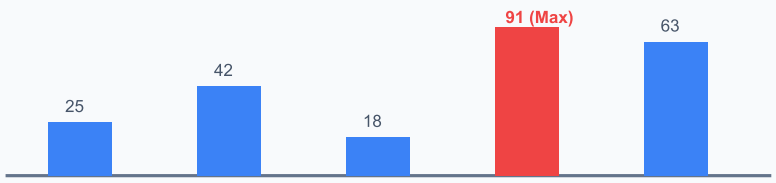
スタイル設定 (Style)

`fillStyle = '#2563eb'`

塗りのカラーやグラデーションを描画命令を実行する直前にセットします。

INTEGRATION

配列データとキャンバス描画の融合



配列インデックスと X 座標

X 座標 = **offset + i * barWidth** で各要素を横方向に均等配置します。

配列数値と Y 座標・高さ

高さ **h = array[i]**, Y 座標 = **canvasHeight - h** （原点が左上のため反転補正）。

ループによる全要素自動出力

for ループ内で **fillRect()** を連続呼出し、配列全体を一括描画します。

VISUAL HIGHLIGHTING

最大値要素のグラフィック強調表示

```
        for (let i = 0; i < array.length; i++) { if (array[i] === max) {
ctx.fillStyle = '#ef4444'; // 最大値要素を赤色で強調 } else { ctx.fillStyle =
'#3b82f6'; // 標準描画色は青色 } ctx.fillRect(i * 60 + 40, 300 - array[i], 40,
array[i]); }
```

アルゴリズムと描画の動的同期

探索アルゴリズムで得た max 値と条件分岐を組み合わせることで、対象の棒グラフのみ配色を動的に変化させます。

応用思考：最小値の同時強調

最小値（min）も同時に探索・保持することで、else if (array[i] === min) により別色（緑など）での視覚強調が可能です。

SUMMARY & ASSIGNMENT

まとめと今週の実習課題

データ走査の「論理的思考（ロジック）」と Canvas 描画の「視覚表現」を融合させることで、直感的かつ実用的なデータ可視化プログラムを構築できます。

チャレンジ課題

8つの数値要素を持つ配列を入力とし、各要素の棒グラフをキャンバス上に描画した上で、**「最大値要素を赤色」「最小値要素を緑色」**に自動色分け表示するプログラムを構築し、結果画面とソースコードを提出しなさい。